SwiftCNS — PRD - 03/27/2026

SwiftCNS — Product Requirements Document

This document describes SwiftCNS as a product: the problem it addresses, how it works for users and buyers, what is in scope, how pilots succeed, and what remains to harden for scale. It is written for planning and alignment without pointing to internal engineering artifacts.

---

1. Product identity and vision

SwiftCNS is positioned as **the learning system for innovation**: a place where **ideas, assumptions, experiments, and signals** become **confident decisions** through structure, visibility, and assistance—not through scattered notes and ad hoc tests alone.

CNS means **Central Nervous System**. The name is intentional: a venture must **sense** the world (inputs, documents, conversations, market feedback), **interpret** that information into explicit beliefs and tests, and **act** with coordinated next steps. SwiftCNS is meant to be that coordinating layer—so learning is **systematic**, **shared**, and **compounding** instead of trapped in individual inboxes or slide decks.

Core promise. SwiftCNS helps innovators **increase both the speed and the quality of learning**. Speed comes from reducing friction between “what we think” and “what we ran,” and from assistance that proposes next tests and synthesizes outcomes. Quality comes from **evidence**, **traceability**, and **validation discipline**—so teams do not mistake activity for insight.

Strategic thesis. Innovation is increasingly a **learning race**, not only a **build race**. Tools and automation have made it cheaper to ship and to generate options; they have not removed the cost of **being wrong** about what customers value, what will convert, or what will scale. The limiting factor for many teams is therefore **learning velocity**: how quickly uncertainty turns into **documented**, **reviewable**, **actionable** knowledge—and whether that knowledge is visible to the people who must coach, fund, or govern the work.

---

2. Market context and the problem SwiftCNS solves

2.1 Why innovation fails or stalls

Public and industry commentary frequently highlights harsh outcomes for new products and early-stage companies: many launches underperform expectations, large amounts of capital are spent on bets that do not pay off, and a substantial share of startup failure is attributed to **lack of**

market need or misread demand. SwiftCNS is aimed at the **operational** layer behind those headlines: even when teams are busy and intelligent, they often **do not learn fast enough** or **learn in ways that do not compound**.

SwiftCNS assumes the root issue is often **not** a shortage of ideas but **slow validation**, **weak learning loops**, and **poor institutional memory** about what has been tried and what was learned.

2.2 The moment: speed without clarity

Teams **ship, test, and launch faster**, but still struggle to answer: what to **double down** on, **change**, or **stop**. **Compressed build cycles**, cheaper experimentation, and **AI-assisted building** increase output but **do not remove the cost of being wrong**—so the bottleneck moves to **how fast teams learn** and compound signal.

2.3 Hidden problems inside teams and programs

- **Ad hoc experimentation** — Under pressure, teams skip structured cycles or run tests that cannot be compared week to week.
- **Fragmentation** — Signals live across chat, documents, analytics tools, and people's heads; no **single shared memory** of beliefs, tests, and conclusions.
- **Validation gaps** — Desirability, feasibility, and viability stay fuzzy until late.
- **Slow insight cycles** — Designing tests and turning results into **decision-ready** summaries can lag by **weeks**.

For **innovation programs**, fragmentation appears at **portfolio** scale: mentors and staff lack a consistent view of each venture's **theory**, **active tests**, and **learnings**.

2.4 The theoretical anchor: learning as the job of the startup

A widely used definition treats a startup as a **temporary organization designed to search for a repeatable and scalable business model**. If that is true, then **speed of learning** is the **constraint** on finding that model. SwiftCNS is built around: **accelerated learning → better-informed decisions → faster convergence** on what works.

2.5 What "good" looks like at elite operating maturity

Sophisticated organizations run **assumptions → experiments → signals → decisions**, often with very high experiment velocity and short setup time. For most organizations those systems remain **manual**, **fragmented**, **expert-dependent**, and **hard to operationalize**. SwiftCNS aims to **productize** that operating discipline.

2.6 Product thesis: structured learning loop

SwiftCNS expresses the loop as: **assumptions and hypotheses → experiments → learnings → insights and recommended actions**. **Signals** are the evidence stream that becomes structured learning inside the system.

Without a system like SwiftCNS	With SwiftCNS
Assumptions stay implicit	Assumptions **explicit**
Documents and results scatter	Learnings **centralize** and **compound**
Experiments are ad hoc	Experiments **structured** and traceable
Decisions skew opinion-driven	Decisions **evidence-backed**

Four capabilities: Innovation copilot (conversational assistance aligned to discovery, design, planning, synthesis); **automate and accelerate** (less overhead from belief to logged outcome); **shared memory** (durable workspace for conversations, documents, entities, lineage); **orchestrated intelligence** (coordinated assistants with retrieval over project knowledge).

Closing thesis: Innovation is a **learning race**, not only a **build race**—SwiftCNS exists to make that learning **systematic**.

---

3. Scope: what SwiftCNS contains

3.1 Core concepts

Workspaces — Personal, shared, or enterprise-style boundaries for who belongs where.

Projects — A venture or initiative; holds structured artifacts and conversations.

Conversations — Persistent assistant threads with stored messages and tool-driven outputs.

Documents — Uploads that ground chat and structured workflows (e.g. learning synthesis).

Assumptions — Explicit beliefs, often with evidence and importance ratings.

Hypotheses — Testable claims, often linked to assumptions; themes such as desirability, feasibility, viability.

Experiments — Structured plans and records: what is tested, success criteria, operational constraints, lifecycle status, notes.

Learnings — Observations, data reliability, linked insights, recommended actions; workflow status (e.g. draft vs validated).

Insights — Durable conclusions with implications, confidence, tags; clear lineage from learnings.

Sharing — Invitations with roles (e.g. viewer, editor, member) for mentors, staff, or collaborators.

Administration — Platform or tenant operators, separate from end-user innovation workflows.

Lineage — Links between entities so reviewers can trace belief → test → outcome.

3.2 Typical experience

Dashboard summarizing projects, conversations, experiments, learnings, insights, and documents; methodology libraries (patterns, experiment ideas); project hub; **notebook** layout pairing **sources** with **assistant**; structured views for experiments, hypotheses, learnings, and insights.

3.3 Intelligence (conceptual)

Streaming assistant responses; **tools** that produce structured outputs users can open and edit; **retrieval** over project documents so answers reflect **this team's** context; specialized assistants for parts of the loop with a **coordinator** managing flow and persistence.

```
flowchart LR
  subgraph people [People]
    FounderMentorStaff[Founders_mentors_program_staff]
  end
  subgraph experience [SwiftCNS_experience]
    Home[Dashboard_and_projects]
    Notebook[Notebook_chat_and_sources]
    Structured[Structured_EMS_views]
  end
  subgraph platform [Platform_services]
    Data[Accounts_projects_entities_documents]
    Chat[Streaming_assistant_and_tools]
    Memory[Shared_persistence_and_search]
  end
  FounderMentorStaff --> Home
  Home --> Notebook
  Home --> Structured
  Notebook --> Data
  Notebook --> Chat
  Chat --> Memory
  Structured --> Data
  Memory --> Data
```

4. Value by audience

Startups — Name risk early, test before overbuilding, capture learning durably; adoption works best when programs **coach the loop**, not only chat.

Incubators, accelerators, city programs — Portfolio visibility, accountability, richer mentor conversations; procurement often moves toward seats, cohort licenses, or enterprise deployment with security and reporting expectations.

Adjacent segments — Consultancies (strong fit); corporate labs (strong fit, enterprise gates); research / tech transfer (medium); solo coaches (medium). SwiftCNS differentiates where buyers want **rigor and reporting**, not generic chat-only AI.

---

5. Pedigree and proof (qualitative)

The organization describes **years of practice** helping teams validate and ship; SwiftCNS is that practice **productized**. Internal case narratives (e.g. structured experiment-led pivots in estimation / AI-assisted workflows) should be used with **customer permission** and **verifiable** figures when cited externally.

---

6. Maturity and stage

Working stage label: Late beta / early commercial — the core product can support real pilots and paying design partners; **buyer-ready packaging**, **program-level analytics**, and **enterprise hardening** are still catching up to the depth of the innovation workflow.

6.1 Maturity across dimensions

Dimension	Current state (summary)	Maturity (for planning)	Implications
Core innovation loop	Assumptions through experiments, learnings, insights, sharing, and chat are usable end-to-end in a real workspace	**High**	Pilots can test the **full thesis**, not a slideware subset

Assisted workflows	Streaming assistant, tools, document-grounded responses, multi-step orchestration	**High**	Differentiator is live; quality and cost need ongoing tuning
Collaboration	Workspaces, roles, shared projects/conversations	**Medium–high**	Sufficient for startups + mentors; cohort **oversight** views may be thin
Program / portfolio visibility	Per-project richness strong; **aggregated cohort** reporting often manual or roadmap	**Low–medium**	Track B pilot KPIs may require **CS + exports + reviews** until dashboards ship
Identity & access	Functional auth; enterprise expectations (SSO, SCIM, audit) may be partial	**Medium**	City and corporate buyers will probe **early**
Production presentation	End-to-end hosting may still signal “pre-GA” in naming or docs	**Medium**	Affects trust in security reviews and procurement
Methodology adoption	Powerful structure exists; **discipline** still depends on coaching	**Medium**	Pilot success ties to **customer success**, not only features

Data for KPIs	Entity timestamps and counts can support many pilot KPIs; some need **definitions + exports + surveys**	**Medium**	“Strong data” needs **owners and process** (see §7)

6.2 Product and technical strengths (what is already credible)

Area	What is in place	Why it matters for pilots
Structured entities	Rich experiment schema, learnings with observations and recommended actions, insights, lineage	Supports **evidence-backed** and **actionable** KPIs
Persistence & APIs	Durable messages, entities, pagination, incremental updates	Enables **auditable** pilot measurement, not screenshots
Async synthesis	Long-running learning generation where applicable	Reduces manual write-up burden during busy pilot weeks
Real-time UX	Responsive streaming conversation experience	Drives **activation** and weekly/biweekly ritual completion
Document grounding	Uploads and retrieval scoped to project context	Improves **quality** of synthesis and mentor visibility

6.3 Gaps and risks (what will constrain scale)

Gap / risk	Symptom for users or buyers	Pilot mitigation (until shipped)

Chat-only usage	High conversation volume, **low** EMS entity creation	CS playbooks, mandatory checkpoints, mentor expectations in Track B
Weak cohort rollup	Program leads cannot see “whole cohort” without manual work	Weekly **export** + office hours; narrow pilot N for Track B
Enterprise checklist	SSO, residency, SLAs, formal audit	Start with Track A and smaller hubs; document roadmap dates
Metric definitions	Disagreement on “evidence-backed” or “actionable”	**Written rubric** + calibration session with staff (see §10)
Inference cost volatility	Surprise bills if pilots are document-heavy and chat-heavy	Caps, monitoring, model routing, pilot **usage** expectations in SOW

6.4 Readiness for the two pilot tracks

Track	Product readiness	Operational reliance
A — Startup	**Ready** to test operating rhythm and learning velocity if CS enforces ritual	Founder onboarding + **primary KPI** baseline capture in week one
B — Cohort	**Ready** for learning capture per startup; **program visibility** KPIs may need **process + partial tooling**	Staff review cadence, mentor onboarding, **70%+** mentor participation target

7. Resources and capacity (development, pilots, and strong data)

This section answers: **who and what budget** are needed to **ship** the product during pilots, **run** pilots credibly, and produce **defensible metrics**—not only “more engineers.”

7.1 Roles and capacity — product development (concurrent with pilots)

Role / function	Minimum viable (pilot period)	Recommended (parallel tracks + credibility)	Primary responsibilities
Product owner / domain lead	**0.5–1.0** FTE	**1.0** FTE	Pilot success criteria, rubrics for evidence/actionable, roadmap tradeoffs, buyer narrative alignment
Engineering (full-stack or FE+BE pair)	**1–2** FTE	**2–3** FTE	Reliability fixes, cohort/admin surfaces, exports, instrumentation hooks, SSE hardening
Applied AI / ML engineer	**0.25–0.5** FTE	**0.5–1.0** FTE	Retrieval quality, evals, latency/cost, prompt/tool behavior during live pilots
Design (product + content)	**0.25** FTE	**0.5** FTE	Onboarding, ritual UX (weekly/biweekly updates), program-facing views

DevOps / SRE / security	**0.25** FTE	**0.5–1.0** FTE	Environments, observability, incident response, access reviews for B2B
QA or test owner	**0.1–0.25** FTE (often shared)	**0.25–0.5** FTE	Regression before demo weeks; data integrity checks for metrics

FTE = full-time equivalent; fractional roles are common at this stage.

7.2 Roles and capacity — pilot execution (Track A + Track B)

Role / function	Minimum viable	Recommended	Responsibilities tied to pilot KPIs
Pilot / customer success lead	**0.5** FTE	**1.0** FTE	Week-one **activation**, ritual nudges, unblockers; owns **update completion** rates
Onboarding & training	Bundled with CS	Dedicated hours weekly	Mentor + staff training for **Track B** (G6 mentor participation)
Executive / founder sponsor	**Visible** weekly	Active in partner reviews	Unblocks access, pricing, and scope; signals seriousness to partners

Partner single point of contact	1 named per pilot org	Same	Scheduling, expectations, escalation; reduces dropout

7.3 Resources and capacity — strong data (measurement integrity)

Strong pilot data requires **definitions**, **collection**, and **governance**—not only database fields.

Workstream	Owner (typical)	What "good" looks like	Capacity hint
KPI dictionary	Product + CS	One-page definitions for **evidence-backed**, **actionable**, **visible progress**, **classifiable**	**2–5 days** upfront + 1h/week calibration
Source of truth	Eng + product	Agreed mapping: which entities/events feed each KPI (e.g. median cycle time timestamps)	**Small eng spike** per KPI; avoid spreadsheet sprawl
Exports / dashboards	Eng	Program lead can answer "who is stuck?" without SQL	Prioritize for **Track B** if UI lags
Survey + interview protocol	CS or PM	End-of-pilot **confidence** metrics are structured and comparable	Template + **30–45 min** interviews × N

Weekly pilot review	PM or CS + founder sponsor	**Top five** KPIs + gate review; **outcomes and exceptions logged** (who is off-track, why, agreed fix)	**60 min**/week
Data quality checks	Eng or analyst	Spot-check: empty learnings, mis-linked lineage, backdated timestamps	**Automated** where possible post-pilot

7.4 Budget buckets (indicative, not a quote)

Bucket	Scales with	Notes
LLM inference	Active users × conversation depth × document retrieval	Largest variable; set **guardrails** with pilot partners
Embeddings & storage	Document volume per project	Grows with cohort document uploads
Infrastructure	Environments, logging, backups	Stepped increase for enterprise RFPs
Pilot-specific	Trainings, travel for kickoffs, incentives (if any)	Keep explicit in pilot SOW
Tools	Analytics, survey, support ticketing	Small but non-zero

7.5 Minimum “package” to run both tracks without fooling yourself

If you only have…	You can still…	You risk…

Eng + CS, no metric owner	Run pilots	**Inconsistent** scoring; weak learnings from the pilot
No exports / cohort view	Run Track A strongly	Track B **visibility** KPIs become **subjective** only
No written rubric	Hit numeric targets	**Gaming** or ambiguous “evidence-backed” rates
<0.5 CS FTE	Onboard startups	**Activation** and **ritual** gates fail → inconclusive product read

---

8. Strategic priorities

Product — Prove the closed loop; **program-lead visibility** for hub buyers; guided flows to reduce chat-only usage; lightweight **weekly/biweekly update** support for pilot rituals; **roadmap** explicit **decision / commitment** (or structured next-step) tracking so **actionable learning** KPIs rely less on manual attestation.

Engineering — Reliable streaming for demos; retrieval quality and cost controls; aligned customer-facing environment and identity.

Go-to-market — Lead with discovery-heavy segments; expand corporate as enterprise requirements are met.

Capacity — Staff and budget for development, pilot operations, and **measurement integrity** are specified in **§7**; strategic tradeoffs should be explicit (e.g. defer Track B scale until cohort rollup exists).

Path — Run **two pilot tracks** (§10) in parallel or staggered with a **capped** partner count; use cohort track to pressure-test visibility.

---

9. PRD process success criteria

Leadership agrees on **stage**, **wedge buyer**, and pilot emphasis. The organization can **measure** structured adoption—not only active use—with **owners**, **targets**, and **week-one activation** plans. Resourcing matches **§7** (development, pilots, and data), not only engineering headcount.

---

10. Pilot program: two tracks, success criteria, and KPIs

SwiftCNS runs **two complementary pilots**: **Track A — Startup Pilot** (weekly rhythm, business KPI bridge) and **Track B — Innovation Program Cohort Pilot** (biweekly cohort rhythm, mentor and staff adoption, visibility and support).

10.1 Critical vs important KPIs (pilot scorecard)

Critical KPIs decide whether the pilot is **valid** (conclusions about SwiftCNS are trustworthy). **Important** KPIs explain **why** something worked or failed and whether the **full** value proposition is emerging.

Unified top five (review weekly on either track)

#	KPI	Track nuance	Detailed description
1	**Activation**	A: every pilot startup; B: high share of cohort	Teams are not just invited—they **use** SwiftCNS for real work by **week one**: accounts live, one clear **pilot objective** declared, and initial **assumptions/experiments** (and staff/mentor access for B) exist so downstream metrics reflect

			product usage, not empty tenants.
2	**Ritual adherence**	A: **weekly**; B: **biweekly** at cohort touchpoints	The agreed **operating rhythm** actually happens: each expected update window has a completed check-in (in-app or captured via an agreed template imported into SwiftCNS). Numerator and denominator for “expected updates” are **fixed at kickoff** so completion rates are comparable week to week.
3	**Median learning cycle time**	Same definition both tracks; B also cohort-aggregated	For each **completed** learning, measure **calendar days** from **assumption created** to **learning documented**; report the

			median across all such cycles in the pilot window (median dampens one very long experiment). Shows whether uncertainty is turning into captured learning **faster** than baseline.
4	**Evidence-backed learning rate**	Same idea both tracks; cohort applies across all startup learnings	**Quality** gate: each learning must meet a **written rubric** for “evidence” (e.g. linked experiment outcome, customer quote, metric screenshot summary, document excerpt). The rate is *evidence-qualified learnings ÷ all learnings* so teams cannot pass the pilot on opinion-only notes.

5	**Outcome bridge**	A: business KPI or causal clarity; B: visibility + support taxonomy	**Track A:** Tie learning activity to **one primary growth KPI** chosen at kickoff (movement over the pilot, or a defensible **directional** story if the KPI is immature). **Track B:** Program staff can see **current** progress per startup **and** consistently label who is progressing, stalled, or needs intervention—otherwise hub value is not demonstrated.

Critical gates (pass/fail for a credible pilot)

Gate	Track A — Startup	Track B — Cohort	Why critical	Detailed description
G1 — Activation	**100%** by week one with objective + initial entities	**≥80%** cohort by week one	Weak activation invalidates downstream metrics.	**Week one** ends with: onboarding done; **one** named pilot objective; at least **one assumption and one**

				experiment logged per activated startup; mentors/staff invited where required (B). Exclusions (e.g. dropout) are documented so denominators stay honest.
G2 — Ritual	**≥75%** weekly updates	**≥75%** biweekly updates	Proves **operating cadence**, not one-off trials.	**Completed update** = required fields submitted for that period (e.g. progress vs objective, tests run, blockers, next actions) per the pilot playbook. Count *completed ÷ expected* for each period; failing periods trigger a **root-cause**

				note (tooling vs capacity vs misfit).
G3 — Speed of learning	**Median learning cycle time** measured and comparable	Same, aggregated	Core **speed-of-learning** thesis.	Compute cycle length only when both endpoints exist in SwiftCNS (assumption timestamp, learning timestamp). Exclude incomplete cycles from median. Establish **week-1 baseline** snapshot where possible; compare end-of-pilot median to baseline or to a pre-agreed benchmark.
G4 — Learning quality	**≥80%** evidence-backed learnings	Same cohort-wide	Prevents “learning theater.”	Apply the **same evidence rubric** to every learning;

				disagreements are resolved by **dual review** (e.g. PM + CS) on a sample weekly. “Evidence-backed” means an external reader can see **what was observed** and **why it supports** the learning statement.
G5 — Business / buyer bridge	Primary KPI movement **or** clearer causal story	**≥80%** visibility coverage **and ≥80%** startups **classifiable** for support	Links product to **startup** (A) and **hub** (B) outcomes.	**A:** Document baseline and end value for the chosen KPI; if flat, require a **causal brief** linking learnings to what was validated/invalidated and what changed in the plan. **B: Visibility** = up-to-date

				experiments/learnings visible without chasing side channels; **classifiable** = each startup tagged progressing/stalled/needs-support with **one-sentence rationale** at each touchpoint.

Track B only

Gate	KPI	Target	Why critical	Detailed description
G6 — Mentor continuity	Active mentor participation	**≥70%** of assigned mentors using SwiftCNS	Without mentors in-system, cohort model and context between sessions break down.	**Active** means: logged in during the pilot, viewed assigned startups' **current** work at least **once between cohort sessions**, and used SwiftCNS for

				prep or follow-up (comment, status view, or shared artifact)—not only email. Denominator = mentors assigned to pilot startups at kickoff; numerator adjusted only with written excuse (e.g. mentor replaced).

Important KPIs (diagnostics)

Theme	Track A	Track B	Detailed description
Volume	Assumptions **5–10**; experiments **6–10**; learnings **6–10**; visibility coverage **≥80%**	**2–4** experiments and learnings **per active startup**; assumptions per startup for rigor	**Volume** confirms the loop is **moving**: enough beliefs are named, enough tests are run, enough outcomes are captured—not just one hero experiment. **Visibility coverage** checks that the

			work the team agrees is “in scope” for the pilot objective actually **lives** in SwiftCNS (not parallel spreadsheets).
Usefulness	**Actionable learning rate ≥60%**	**Actionable learning rate ≥60%**	**Product reality:** SwiftCNS today does **not** offer a dedicated **decision log** or single “decision” entity, so “actionable” must be scored with **observable proxies** (see §10.5). **Operational definition:** a learning counts as actionable if **any** of the following exist in-system within an agreed window (e.g. 14 days): **recommended actions** populated on the learning, a **new or materially updated experiment**

			clearly tied to that learning, a **new insight** created from that learning, a **hypothesis/assumption status change** reflecting the conclusion, or (for pilots) a **CS-reviewed attestation** in the pilot scorecard when none of the above apply but a concrete next step was demonstrably taken offline. The rate measures **motion after learning**, not prose quality.
Human proof	Founder/operator confidence (majority positive)	Staff, mentor, founder satisfaction (majority positive)	Structured **end-of-pilot** survey plus optional short interviews. Questions map to clarity ("what to double down / change / stop"), burden ("time well spent"), and **mentor/staff** continuity ("I had

			context between sessions”). Majority = >50% positive or top-two box on Likert, defined in the survey spec.
Portfolio	—	**Majority** of active startups with KPI movement or validated directional learning	For each startup, judge whether **business-facing** progress occurred: movement on its chosen focus KPI **or** a **validated** directional conclusion (“we will not pursue X”) backed by linked learnings. Prevents cohort success from being purely operational if no startup actually learns toward decisions.

Practice — Weekly pilot review = top five + any critical gate at risk. **Expand** pilots only if **all critical gates** pass or there is a documented exception with root-cause analysis. **Important** KPIs drive retros (definitions of evidence/actionable, training, UX).

10.2 Track comparison

Dimension	Track A — Startup	Track B — Cohort

Actors	Founders, operators	Startups, mentors, program staff
Cadence	**Weekly** updates	**Biweekly** at touchpoints
Success layers	Adoption; learning velocity/quality; business progress	Adoption; learning velocity/quality; **cohort visibility & supportability**; startup progress; program value
Typical cycle	Often **4–8 weeks** (confirm in agreements)	**8 weeks**, one **active priority** per startup
Proof	KPI + founder confidence	Majority startups progress or clarity; staff/mentor satisfaction

10.3 Track A — Startup Pilot

Question: Is SwiftCNS in the **operating rhythm**? Is learning **faster and clearer**? Does it connect to **measurable progress** on the chosen growth objective?

Core KPIs (internal)

KPI	Target (pilot)	Detailed description
Startup activation rate	**100%** by week one (objective + initial assumptions/experiments)	All pilot startups meet **G1** by end of week one: live workspace usage, declared **pilot objective**, and initial **assumptions + experiments** logged so the pilot measures real adoption, not invites sent.

Weekly update completion	**≥75%**	Of **expected** weekly check-ins defined at kickoff, the share **submitted on time** with required fields (progress, tests, learnings, blockers, next steps). Late submissions follow an **exception log** so the metric stays honest.
Assumptions captured	**5–10** meaningful	Count assumptions that are **specific**, **testable in principle**, and tied to the pilot objective (not generic platitudes). “Meaningful” is calibrated with CS using **two worked examples** at kickoff.
Experiments logged	**6–10**	Experiments have a **clear intent**, **success criteria**, and **status** updates; each maps to at least one assumption or hypothesis. Proves the team is **running** the loop, not only documenting intent once.
Learnings documented	**6–10**	Learnings are recorded **after** experiments complete or materially advance, with

		observations and (where applicable) links to evidence. Count only learnings **created during** the pilot window unless a pre-pilot backlog rule is explicitly agreed.
Median learning cycle time	Baseline → improve	Median days from **assumption created** to **learning created** for **completed** cycles; excludes open assumptions with no learning yet. Used as the primary **speed-of-learning** indicator for the startup track.
Evidence-backed learning rate	**≥80%**	Share of learnings that satisfy the **evidence rubric** (linked artifacts, metrics, customer data, experiment results). Reviewers spot-check weekly to keep scoring consistent across founders.
Actionable learning rate	**≥60%**	**Do not require a "decision" record**—the product does not yet track decisions as first-class objects. Score using **in-product proxies**: learning has **recommended actions** filled, or a **linked**

		follow-up experiment / **new insight** / **clear lineage** showing the next test or conclusion, or a **documented status change** on a related hypothesis or experiment within the agreed window. If none apply, pilots may use a **binary CS checklist** per learning (Y/N + one-line note) so the metric stays measurable without inventing data. **Roadmap:** explicit decision or “next step” logging would tighten this KPI and reduce manual review.
Visibility coverage	**≥80%** of active growth work on objective visible	Compare **known** active growth initiatives for the objective (agreed in kickoff) to what is represented in SwiftCNS (experiments, learnings, linked docs). Unmapped work is listed explicitly so coverage cannot be assumed at 100% by default.
Primary growth KPI	Positive movement or clearer understanding	The **one** KPI named at kickoff (e.g. activation,

	if immature	revenue, conversion, pipeline): capture **baseline**, weekly notes, and **end value**. If flat, require a **short causal memo** tied to learnings explaining what was falsified or what blocked movement.
Founder/operator confidence	Majority positive	Standardized **exit survey** (and optional 15–20 min interview) on clarity of what works, what to do next, and whether SwiftCNS reduced thrash; "majority" defined in the survey spec.

Median learning cycle time — Median of days from **documented assumption** to **documented learning** (not mean, to resist outlier experiments).

Success (all true): Adoption (week one + **≥75%** weekly updates); Learning (meaningful assumptions, **≥6** experiments, **≥6** evidence-backed learnings, **majority actionable** per the **proxy definition** in §10.5); Visibility (most work visible; clearer picture of tests and learnings); Outcome (KPI movement **or** materially clearer evidence).

External-friendly labels: Time to learning; experiments run; learnings captured; actionable insights; visibility; progress on primary KPI.

10.4 Track B — Innovation Program Cohort Pilot

Question: Is learning captured **across the cohort**? Is **uncertainty → learning** faster/clearer? Do staff get **support-ready visibility**?

Core KPIs (internal)

KPI	Target (pilot)	Detailed description

Cohort activation rate	**≥80%** by week one	Share of **enrolled** pilot startups that meet **G1** by week one (objective + initial entities + correct sharing for staff/mentors). Dropouts before week one are excluded only if documented per playbook.
Biweekly update completion	**≥75%** per cycle	For each cohort touchpoint, **expected** startup updates ÷ **completed** updates, using the same field requirements as Track A but on a **biweekly** cadence aligned to program sessions.
Active mentor participation	**≥70%**	Meets **G6**: mentors **use** SwiftCNS between sessions for assigned startups; participation is **observable** (login + substantive view or comment), not claimed informally.
Experiments per active startup	**2–4**	Meaningful experiments **per startup that remains active** in the pilot (excludes shells). Normalizes volume so a large cohort cannot hide weak per-team

		usage behind aggregate counts.
Learnings per active startup	**2–4**	Documented learnings tied to **completed or advanced** experiments per active startup; encourages **signal per venture**, not only program-level activity.
Median learning cycle time	Baseline → improve	Median of per-startup medians (or pooled cycles, per agreed statistics note) from assumption → learning, so cohort speed can be compared **across** ventures with different focus areas.
Actionable learning rate	**≥60%**	**Same proxy definition as Track A** (recommended actions, linked experiments/insights, status changes, or CS attestation when needed). Scored **across all cohort learnings** so hub-level rates stay comparable; optionally report **per-startup** distribution to catch weak teams.
Program visibility coverage	**≥80%** startups with current progress visible	For each startup, staff can see **up-to-date** objective, active

		experiments, and recent learnings **without** offline chase. “Current” means updated within the **agreed freshness window** (e.g. before each session).
Support intervention identification	**≥80%** classifiable at touchpoints	At each scheduled review, each startup receives **one** status: progressing / stalled / needs support, with a **one-line reason** tied to visible artifacts in SwiftCNS where possible.
Startup objective progress	**Majority** of active startups	Each active startup shows either **KPI movement** on its focus metric or a **validated directional** outcome (e.g. pivot, deprioritization) supported by linked learnings—not ambiguous “still exploring.”
Program confidence / satisfaction	Majority positive	Survey **staff, mentors, and founders** on visibility, continuity between sessions, and whether SwiftCNS improved **structure** of learning; same majority

		rule as Track A, with role-specific questions.

Additional internal metrics (cohort)

KPI	Detailed description
Assumptions per active startup	Count **meaningful** assumptions per startup tied to its **single active priority** for the cycle; ensures each venture surfaces uncertainty, not only experiments without stated beliefs. **Targets** are often agreed per program (e.g. a minimum floor) rather than one global number.
Evidence-backed learning rate (cohort-wide)	Apply the same **evidence rubric** as **G4** to **all** learnings from **all** cohort startups so aggregate quality cannot mask weak teams; report **cohort aggregate** and optionally **per-startup** distribution (e.g. share of startups at or above **80%**).

Success (all true): Adoption (≥80% startups, ≥75% biweekly updates, strong mentor use); Learning (assumptions per priority, experiments, evidence-backed learnings, **majority actionable** per **§10.5** proxies); Program visibility (current view; progressing/stalled/needs-support; mentors informed between sessions); Startup progress (KPI movement or clearer evidence); Program value (staff monitoring, mentor continuity, founder structure).

10.5 Operating notes

Updates may start as **CS-led** rituals before full in-product automation. Define **evidence** and **actionable** in writing for consistent scoring. Track B needs staff to **actually review** the system—**dashboards/exports** reduce friction. Mentors need clear **access** and onboarding.

Actionable learning rate — how to measure it today (no decision log yet)

SwiftCNS does **not** currently provide a unified **decision record** (“we decided to do X”) that can be queried per learning. Until that exists, use this **stacked** definition so the KPI stays **auditable** and **honest**:

Tier	What counts as “actionable”	Notes
A — Strong (in-product)	Learning has **recommended actions** with content, **or** a **new/updated experiment** linked after the learning, **or** a **new insight** created from the learning, **or** a clear **workflow/status** change on a related experiment or hypothesis that reflects the learning	Prefer **A** whenever possible; ties the metric to real objects.
B — Attested (pilot ops)	None of **A** applies, but **CS or PM** marks the learning **actionable** with a **one-line justification** (e.g. “hired for ICP change,” “paused channel per learning”) and optional link to external artifact	Caps **B** at a **minority** of actionable learnings (e.g. ≤30% of numerator) so the pilot does not devolve into subjective scoring.
Excluded	Vague next steps only in chat or only in a weekly doc **not** mirrored into SwiftCNS	Keeps the metric aligned to **system behavior** and coaching.

Roadmap (product): First-class **decision** or **commitment** objects (or structured “next step” fields with required type) would reduce reliance on **tier B** and make this KPI mostly self-service from analytics.

Rubric ownership: Product + CS maintain the **one-pager** referenced in §7.3; calibrate weekly for the first two weeks of each pilot.

10.6 Design-partner commercial shape

A concentrated design-partner phase: **limited slots**, **time-boxed** engagements, benefits such as founder access, advantaged pricing, roadmap input—aligned with the **two-track KPI model** and **eight-week** cohort mechanics where used.

---

11. Confidentiality and claims hygiene

Validate statistics and named stories before fiduciary, RFP, or public use. Obtain **permissions** and **accuracy** review for customer quotes.

End of PRD.